Paradigmas y Lenguajes de Programación III

Trabajo Práctico: “La Web del Terror Accesible”

Auditoría de calidad (ISO/IEC 25010), accesibilidad (WCAG 2.1) y testing de software

Integrantes:
Leandro Acosta
Santiago Romero Baylac
Tomás Romero Baylac

Docente: Dante Sicardi

Fecha de entrega: 10/11/2026

1. Introducción

En este trabajo retomamos el proyecto del Desafío de Halloween, una página para votar disfraces hecha con HTML, CSS y JavaScript en el frontend y PHP con MySQL en el backend. El objetivo era mejorarlo para que cumpla con las pautas de accesibilidad WCAG 2.1 nivel AA, que se pueda usar solo con el teclado y que pase la auditoría de Lighthouse con 100 puntos. Después completamos la ficha de autoevaluación según la norma ISO/IEC 25010.

El código de partida fue el del repositorio del equipo en GitHub. Antes de empezar a cambiar cosas corrimos Lighthouse para ver dónde estábamos parados.

2. Situación inicial

La página principal daba 91 en accesibilidad y las páginas de login y registro daban entre 89 y 92. Los problemas que marcaba Lighthouse eran dos:

- Los botones “¡Votar!” y “Entrar” tenían letra blanca sobre fondo naranja. El contraste daba 2,9:1 y la norma pide como mínimo 4,5:1.
- Las páginas de login y registro no tenían la etiqueta `<main>`.

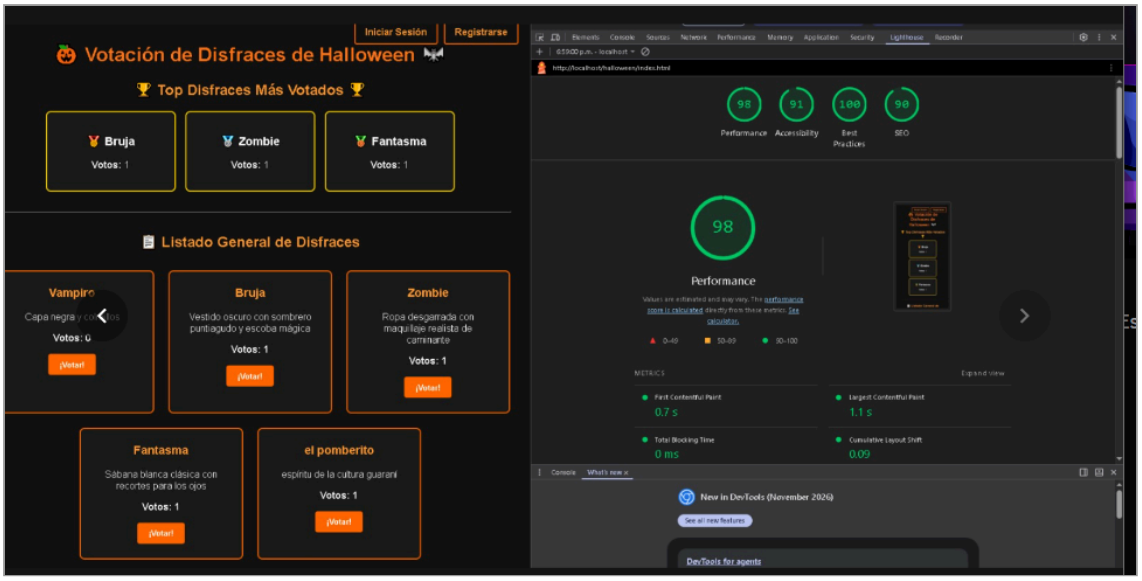


Figura 1. Lighthouse de la versión original (accesibilidad 91).

Al revisar el código para completar la ficha ISO también encontramos otros problemas que no aparecen en Lighthouse pero que afectaban la seguridad y el funcionamiento:

- Las contraseñas se guardaban en texto plano en la base de datos, sin hash.
- Había dos logins distintos: `login.html` mandaba un JSON con el nombre de usuario, pero `api/login.php` esperaba un formulario con el email. Por eso el login de la página accesible no funcionaba.
- Cuando fallaba el voto, `votar.php` le devolvía al usuario el mensaje de error de la base de datos.

- Las tarjetas se armaban con `innerHTML` metiendo directamente los datos de la base, lo que permite inyectar código (XSS).
- Los resultados se mostraban con `alert()` y después se recargaba la página, así que el foco del teclado se perdía.
- El repositorio no tenía el script para crear la base de datos.

3. Cambios de accesibilidad

3.1 Correcciones de Lighthouse

Pasamos todos los colores a variables en un único archivo `css/estilos.css`. Para los botones dejamos el mismo naranja pero con la letra oscura (`#121212`), así el contraste sube a 7:1 y el diseño queda casi igual. Comprobamos el resto de los colores con la fórmula de contraste de WCAG:

Elemento	Colores	Contraste	Mínimo
Botón original	blanco sobre #ff6600	2,94:1	4,5:1 (no cumplía)
Botón corregido	#121212 sobre #ff7518	6,97:1	4,5:1
Título principal	#ff7518 sobre #121212	6,97:1	4,5:1
Nombres de los disfraces	#ffa94d sobre #1e1e1e	8,76:1	4,5:1
Descripciones	#c8c8c8 sobre #1e1e1e	9,96:1	4,5:1
Mensajes de error	#ff8a80 sobre #1e1e1e	7,30:1	4,5:1
Borde de los campos	#8a8a8a sobre #2a2a2a	4,16:1	3:1

Además agregamos `<header>`, `<nav>` y `<main>` en las tres páginas, dejamos un solo `<h1>` por página y pusimos `<label for="...">` en todos los campos de los formularios. Cada disfraz que tiene foto la muestra con un `alt` que describe el disfraz, y a los emojis decorativos les pusimos `aria-hidden="true"` para que el lector de pantalla no los lea.

3.2 Navegación con teclado

Sacamos los `onclick` que estaban escritos dentro del HTML y usamos solamente botones y enlaces nativos, que ya funcionan con Tab y Enter. También hicimos estos cambios:

- Un enlace “Saltar al contenido principal” que aparece con el primer Tab.
- Un borde blanco con sombra naranja usando `:focus-visible`, para que se vea bien dónde está el foco.
- Los botones de votar tienen un texto oculto con el nombre del disfraz, así el lector de pantalla dice “Votar por Zombie” y no solo “Votar”.
- Después de votar, el foco vuelve al mismo botón en lugar de irse al principio de la página.

Probamos el recorrido completo usando solamente el teclado: registrarse, iniciar sesión, votar y cerrar sesión. Este es el orden de tabulación de la página principal:

Tab	Elemento	Foco visible
1	Saltar al contenido principal	Sí
2	Logo “La Web del Terror”	Sí
3	Iniciar sesión	Sí
4	Registrarse	Sí
5 a 9	Votar por Bruja, el pomberito, Fantasma, Zombie y Vampiro	Sí

Para confirmar que no se nos escapaba nada también analizamos la página con la herramienta axe en distintos estados de la página (sin sesión, con un error, con la sesión iniciada y después de votar) y no marcó ningún error de WCAG 2.1 AA.

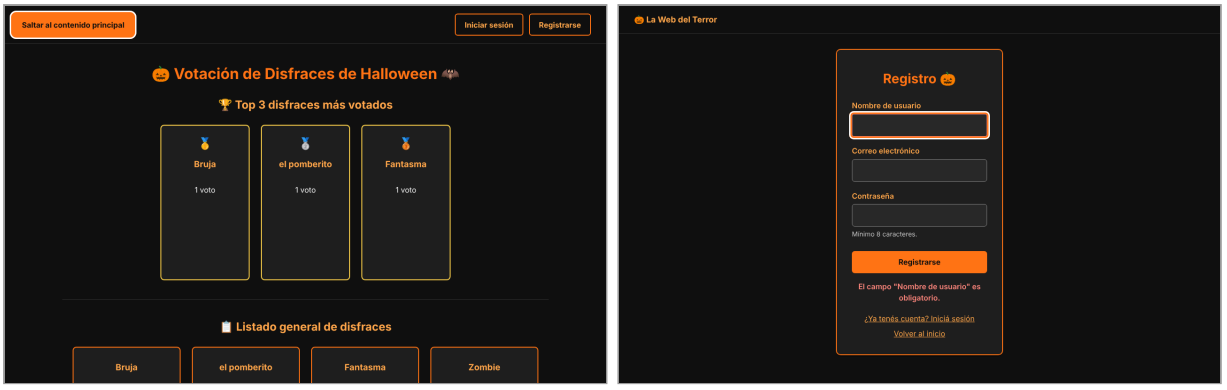


Figura 2. Enlace para saltar al contenido y error de validación en el registro, con el foco en el campo vacío.

3.3 Mensajes de las peticiones fetch()

Reemplazamos los `alert()` por un contenedor con `aria-live="polite"`. Las respuestas de la API se escriben ahí con `.textContent`, así el lector de pantalla las lee sin que el usuario pierda el lugar donde estaba. Esto lo pusimos en una función que usan todas las páginas:

```
export function anunciar(region, mensaje, tipo = 'ok') {
    region.classList.remove('ok', 'error');
    region.textContent = '';
    setTimeout(() => {
        region.classList.add(tipo);
        region.textContent = mensaje;
    }, 50);
}

<div id="aviso" class="aviso" role="status" aria-live="polite"></div>
```

Usar `.textContent` en vez de `innerHTML` también nos sirvió para cerrar el problema de XSS, porque el texto nunca se interpreta como HTML.

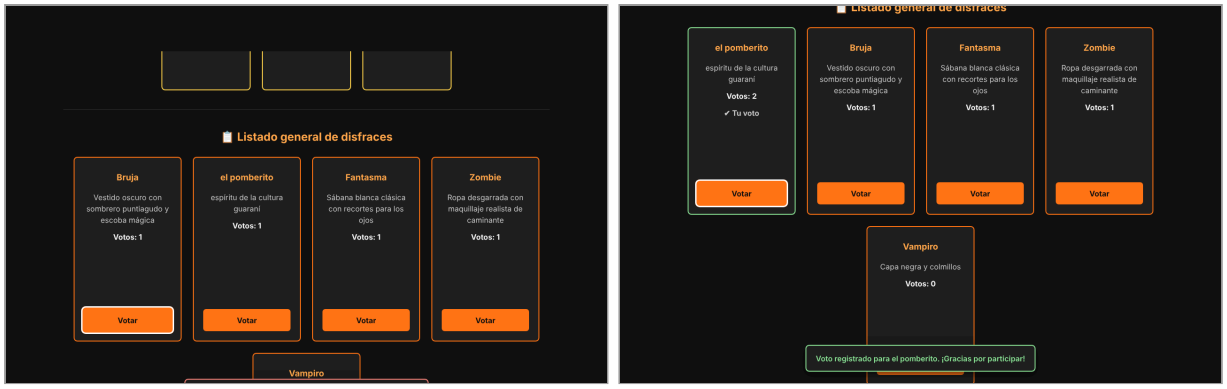


Figura 3. Mensaje cuando se intenta votar sin sesión y mensaje de voto registrado.

3.4 Resultado de Lighthouse

Con los cambios, las tres páginas llegaron a 100 en accesibilidad. Como también corregimos el favicon que faltaba y reservamos el espacio de las tarjetas para que la página no salte al cargar, el resto de las categorías también quedó en 100.

Página	Accesibilidad antes	Accesibilidad después	Rendimiento	Buenas prácticas	SEO
index.html	91	100	100	100	100
login.html	92	100	100	100	100
registro.html	92	100	100	100	100

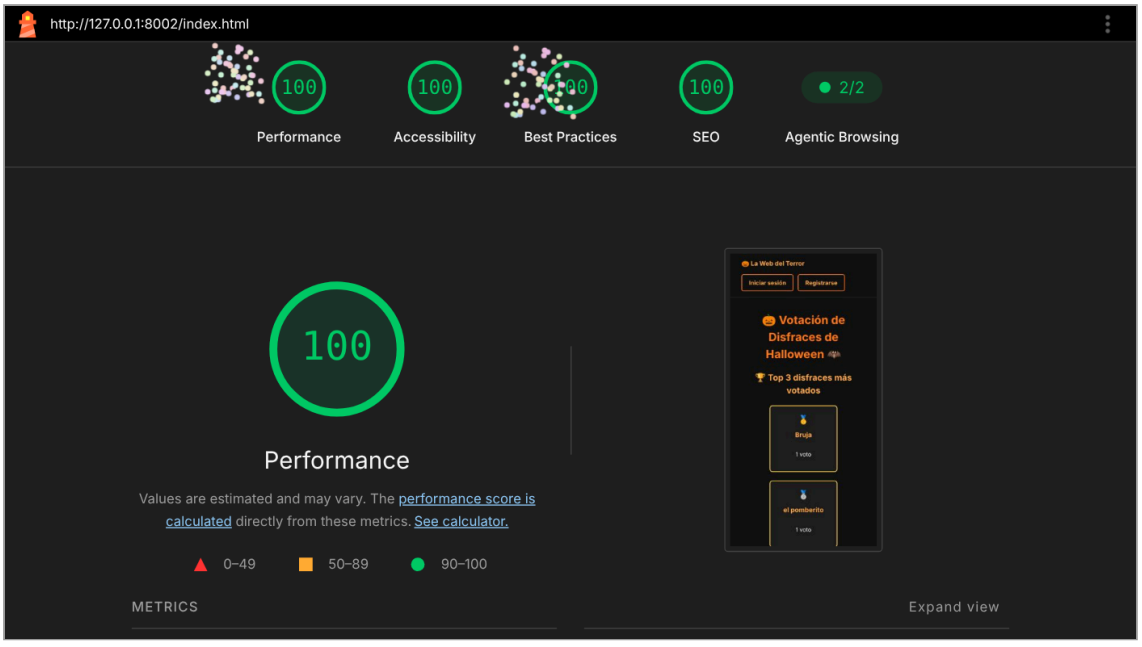


Figura 4. Lighthouse de index.html después de los cambios.

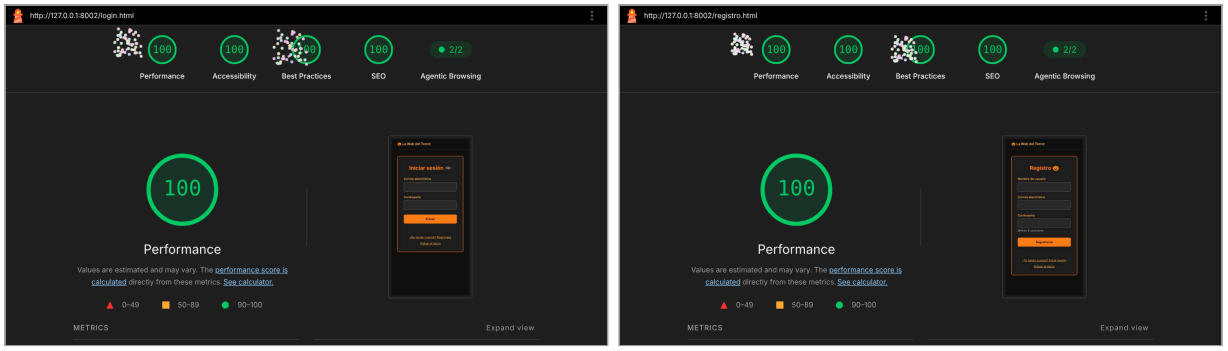


Figura 5. Lighthouse de login.html y registro.html.

4. Cambios en el backend

Para poder completar la parte de seguridad y fiabilidad de la ficha tuvimos que rehacer la API. Los cambios principales fueron:

- El registro guarda la contraseña con `password_hash($clave, PASSWORD_BCRYPT)` y el login la verifica con `password_verify()`.
- Dejamos un solo login, que recibe JSON con email y contraseña. Las páginas PHP con formulario que había antes se eliminaron.
- Todas las consultas usan sentencias preparadas con `prepare()` y `execute()`.
- Cada endpoint tiene su `try/catch` para `PDOException` y devuelve un código HTTP que corresponde a lo que pasó. El error real se guarda con `error_log()` y al usuario solo le llega un mensaje general.
- El voto se hace dentro de una transacción, y en la tabla `votos` el campo `usuario_id` es `UNIQUE`, así nadie puede votar dos veces aunque haga doble clic.
- Al iniciar sesión se regenera el id de sesión y la cookie es `HttpOnly`.

También separamos el código en carpetas: `api/config` para la conexión, `api/lib` para las funciones de respuesta JSON, `js/` para el JavaScript de cada página y `css/` para los estilos. Agregamos el archivo `database/halloween.sql` para poder crear la base.

5. Pruebas de la API

Hicimos un script en PHP (`tests/test_api.php`) que llama a cada endpoint y revisa que devuelva el código HTTP esperado. Se corre con `php tests/test_api.php http://localhost/halloween`.

N°	Prueba	Esperado	Obtenido	Resultado
1	GET disfraces devuelve lista JSON	200	200	OK
2	POST a disfraces.php -> método no permitido	405	405	OK
3	Registro con JSON inválido	400	400	OK
4	Registro con campos vacíos	400	400	OK
5	Registro con email inválido	400	400	OK

6	Registro con contraseña corta	400	400	OK
7	Registro correcto	201	201	OK
8	Registro duplicado -> conflicto	409	409	OK
9	Votar sin sesión -> no autorizado	401	401	OK
10	Login con contraseña incorrecta	401	401	OK
11	Intento de inyección SQL en login	401	401	OK
12	Login correcto	200	200	OK
13	Sesión activa	200	200	OK
14	Votar con id inválido	400	400	OK
15	Votar disfraz inexistente	404	404	OK
16	Voto válido	200	200	OK
17	Segundo voto del mismo usuario -> conflicto	409	409	OK
18	Sesión informa el voto emitido	200	200	OK
19	Cerrar sesión	200	200	OK
20	Votar después de cerrar sesión	401	401	OK

Pasaron las 20 pruebas. El tiempo promedio de respuesta fue de 11,2 ms y el máximo de 70,9 ms. Los pedidos más lentos son el registro y el login, porque BCRYPT tarda a propósito para que sea difícil adivinar contraseñas por fuerza bruta.

6. Ficha de autoevaluación ISO/IEC 25010

Característica	**Evidencia en el código**	**Estado**
1. Adecuación funcional	El backend en PHP guarda usuarios y votos en MySQL usando PDO (`api/config/conexion.php`). Registro, login, votación y cierre de sesión funcionan.	Cumple
2. Eficiencia de desempeño	La API responde en 11,2 ms en promedio según las pruebas. Lighthouse da 100 en rendimiento.	Cumple
3. Compatibilidad	El frontend es HTML y JavaScript sin frameworks y se comunica con el backend solo por `fetch()`. La API devuelve únicamente JSON.	Cumple
4. Fiabilidad	Se capturan las excepciones PDO y se devuelven los códigos 200, 201, 400, 401, 404, 405, 409 y 500. El voto usa transacción con `rollBack()`.	Cumple
5. Seguridad (OWASP)	Contraseñas con BCRYPT, sentencias preparadas contra inyección SQL (probado con `' OR '1'='1`), `.textContent` contra XSS y sesión regenerada al iniciar sesión.	Cumple
6. Usabilidad / accesibilidad	Lighthouse da 100/100 en las tres páginas y todo el sitio se puede usar solo con Tab y Enter.	Cumple

7. Mantenibilidad	El código está separado en capas (conexión, utilidades, endpoints, scripts por página y una sola hoja de estilos) y versionado en Git.	Cumple

7. Conclusión

La característica que más trabajo nos llevó fue la seguridad. Al principio pensábamos que solo había que arreglar los colores para llegar a 100 en Lighthouse, pero al completar la ficha nos dimos cuenta de que no podíamos poner “Cumple” en seguridad si las contraseñas estaban sin cifrar y el login de la página principal ni siquiera funcionaba. Tuvimos que rehacer el registro y el login, y eso también nos obligó a ordenar el manejo de errores y los códigos HTTP. A su vez, los mensajes de error tenían que llegarle al usuario de forma accesible, por eso terminamos usando `aria-live` en todas las páginas.

También aprendimos que Lighthouse no revisa todo. Los problemas de foco al recargar la página o la falta de nombres claros en los botones solo los encontramos probando el sitio con el teclado.

8. Instrucciones para ejecutar

1. Copiar la carpeta `halloween` en `C:\xampp\htdocs` y arrancar Apache y MySQL.
2. Importar `halloween/database/halloween.sql` desde phpMyAdmin.
3. Entrar a `http://localhost/halloween/index.html`.

Repositorio: https://github.com/Leanacostaa/AA-25---Desafio-Halloween/tree/main